

Three Heads Are Better Than One: A Multi-perspective Reasoning Framework for Enhanced Vulnerability Detection

XIN PENG, National University of Defense Technology, China

BO LIN, National University of Defense Technology, China

JING WANG, National University of Defense Technology, China

XIAOLING LI, National University of Defense Technology, China

JUN MA, National University of Defense Technology, China

JIE YU, National University of Defense Technology, China

XIAOGUANG MAO, National University of Defense Technology, China

SHANGWEN WANG*, National University of Defense Technology, China

Automated vulnerability detection is crucial for enhancing software security by identifying potential flaws that attackers could exploit, thereby reducing the reliance on labor-intensive manual code audits. Recent advancements have shifted towards leveraging large language models (LLMs) for vulnerability detection, with techniques like Vul-RAG and VulnSage demonstrating progress through structured prompting and external knowledge integration. However, these approaches typically rely on a single reasoning paradigm, limiting their ability to address the complex and diverse nature of real-world vulnerabilities. To overcome these limitations, we propose REASONVUL, a novel multi-perspective reasoning framework that harnesses cognitive synergy among three specialized LLM agents, each embodying a distinct reasoning mode. The framework begins with independent analyses of the source code, followed by a structured debate mechanism to resolve conflicts through iterative rebuttal and revision, ultimately converging on a collaborative judgment. Evaluated on the PrimeVul dataset, REASONVUL achieves a PairAcc of 40.00% and an F1-score of 72.52%, surpassing the best baseline by 81.24% in PairAcc. Further tests on the JITVUL dataset confirm its generalizability, with a PairAcc of 28.67%. Additionally, we analyzed 542 conflict cases and found that 389 were correctly resolved, highlighting the framework's ability to uncover hidden vulnerabilities through the error-correction mechanism driven by the debate. This work emphasizes the importance of multi-perspective reasoning and collaborative validation in achieving robust and comprehensive vulnerability detection in real-world software systems.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; • **Security and privacy** → **Software security engineering**.

Additional Key Words and Phrases: Vulnerability detection, Large language model

*Shangwen Wang is the corresponding author.

Authors' Contact Information: **Xin Peng**, National University of Defense Technology, Changsha, China, xinpeng@nudt.edu.cn; **Bo Lin**, National University of Defense Technology, Changsha, China, linbo19@nudt.edu.cn; **Jing Wang**, National University of Defense Technology, Changsha, China, wangjing@nudt.edu.cn; **Xiaoling Li**, National University of Defense Technology, Changsha, China, lixiaoling@nudt.edu.cn; **Jun Ma**, National University of Defense Technology, Changsha, China, majun@nudt.edu.cn; **Jie Yu**, National University of Defense Technology, Changsha, China, yj@nudt.edu.cn; **Xiaoguang Mao**, National University of Defense Technology, Changsha, China, xgmao@nudt.edu.cn; **Shangwen Wang**, National University of Defense Technology, Changsha, China, wangshangwen13@nudt.edu.cn.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE190

<https://doi.org/10.1145/3808197>

ACM Reference Format:

Xin Peng, Bo Lin, Jing Wang, Xiaoling Li, Jun Ma, Jie Yu, Xiaoguang Mao, and Shangwen Wang. 2026. Three Heads Are Better Than One: A Multi-perspective Reasoning Framework for Enhanced Vulnerability Detection. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE190 (July 2026), 24 pages. <https://doi.org/10.1145/3808197>

1 Introduction

Software vulnerabilities, as a long-standing and fundamental challenge in the realms of software engineering and security, have become increasingly severe with the rapid expansion of system scale and complexity [3, 30, 39]. Attackers can exploit these vulnerabilities to gain unauthorized access or steal sensitive data, thereby severely threatening the security and reliability of systems. As a traditional security measure, manual code auditing is a labor-intensive process that requires significant time investment and relies heavily on expert knowledge, making it unsuitable for the fast-paced modern development cycle [4, 7, 52, 74]. Therefore, automated vulnerability detection (AVD) has gradually become an important research direction in both academia and industry, aiming to reduce the burden of manual review by automatically identifying potential vulnerabilities [27].

Over the past decade, AVD techniques have primarily included program analysis-based approaches [1] and deep learning-based approaches [49]. Program analysis methods, such as static analysis and symbolic execution, uncover potential vulnerabilities by examining program behavior. While theoretically capable of offering strong coverage and interpretability, these methods are often hindered by challenges such as path explosion and complex control flows, which result in low efficiency and poor scalability when applied to large-scale projects [2, 31]. In contrast, deep learning approaches leverage neural networks to extract feature patterns from historical vulnerability data, demonstrating certain advantages in large-scale scenarios [17]. However, these methods are highly sensitive to the quality of training datasets, and their generalization ability is limited when encountering previously unseen vulnerabilities [22, 61].

Recently, large language models (LLMs) pre-trained on massive amounts of source code and security knowledge bases have opened up a new research paradigm for vulnerability detection [41]. With their exceptional code semantic understanding and contextual reasoning capabilities, LLMs can imitate human experts and identify vulnerabilities that are difficult to detect using traditional methods. Preliminary research [14] shows that with direct zero-shot prompts, LLMs can achieve performance comparable to deep learning-based methods. Moreover, researchers [25, 35, 46] have even explored more sophisticated prompting strategies, such as techniques like Chain of Thought (CoT), to achieve accurate vulnerability detection. Basically, vulnerability detection can be viewed as a process of reasoning. Given a piece of code, the goal is not merely to recognize surface-level patterns or syntactic anomalies, but to infer deeper semantic relationships, potential execution paths, and the presence of unsafe behaviors under specific conditions. This inherently involves a form of reasoning about the program's logic and behavior, whether it is symbolic reasoning, statistical reasoning, or learned reasoning. Motivated by this perspective, we propose to categorize existing vulnerability detection approaches based on the underlying form of reasoning they employ into three fundamental reasoning modes [36]¹:

- **Deductive Reasoning.** This top-down process derives specific conclusions from general principles. In AVD, deductive methods involve applying predefined security rules or standards, such as CERT C [43], to identify violations in code snippets. For example, approaches like SAVul [6] and approaches described in IRIS [26] analyze code by checking for deviations from established secure coding practices, such as unsanitized user inputs leading to SQL injection vulnerabilities.

¹Most other reasoning types can be regarded as variations or combinations of these three modes in cognitive psychology.

These methods excel at detecting well-defined, rule-based vulnerabilities but may struggle with context-specific issues not covered by existing rules.

- **Inductive Reasoning.** This bottom-up process forms general principles from specific observations. The inductive methods analyze historical vulnerability data or known exploits to identify recurring patterns, such as the misuse of specific APIs, and apply these patterns to detect similar vulnerabilities in new code. Existing approaches [28, 65], such as Vul-RAG [10], leverage retrieval-augmented generation (RAG) to match code against known vulnerability patterns, making them effective for detecting common, recurring vulnerabilities but less adept at identifying novel or context-dependent issues.
- **Abductive Reasoning.** This process seeks the most plausible explanation for an observation by reasoning backward from effect to cause. The abductive methods hypothesize potential exploit scenarios, such as authentication bypass or race conditions, and trace back to identify code weaknesses that could enable such outcomes. For instance, VulnSage [76] adopts a reflective perspective to analyze code by reasoning from observed outcomes. These approaches excel in detecting complex, context-specific vulnerabilities that require an understanding of intricate program behaviors. However, they may fail to identify vulnerabilities that deviate from the assumed scenarios, as their effectiveness largely depends on the quality of the hypothesized attack chains.

Despite the promise of individual approaches, we posit that their confinement to a single reasoning mode constitutes a critical limitation [19]. This narrow focus overlooks the complementary strengths of different cognitive strategies, thereby hindering their effectiveness against the multifaceted nature of real-world vulnerabilities [45]. The integration of multiple reasoning paradigms has historically addressed complex problems. A notable case is Louis Pasteur's refutation of the theory of spontaneous generation.² His experiments relied on inductive reasoning, as he generalized from observations of microbial growth; deductive reasoning, as he designed the swan-neck flask experiment to test clear predictions; and abductive reasoning, as he inferred that contamination was best explained by microbes carried in the air. This case illustrates how diverse modes of scientific reasoning can complement one another to yield robust conclusions. Inspired by this, we hypothesize that integrating multiple forms of reasoning could similarly enhance vulnerability detection. To investigate this hypothesis, we conducted an empirical study on the PairVul dataset [10] (detailed in Section 3), applying three reasoning paradigms to the same set of programs. Our results reveal that the sets of vulnerabilities detected by each mode are largely distinct, with significant complementarity among them. These findings suggest that *merely relying on a single reasoning paradigm is insufficient for effectively detecting the complex vulnerabilities present in real-world scenarios.*

Motivated by this observation, we propose REASONVUL, a collaborative framework designed to harness the power of **multi-perspective reasoning** through a process of **cognitive synergy**. The core idea is to assemble experts embodying diverse reasoning paradigms, enabling multi-perspective structured analysis and cross-validation of viewpoints to enhance both the accuracy and reliability of the detection. Specifically, we design three agent roles that simulate security experts specializing in deductive, inductive, and abductive reasoning. During the detection process, the three agents first conduct independent structured reasoning on the source code and produce intermediate conclusions accompanied by explanatory justifications. When their conclusions diverge, a debate stage is initiated. In this phase, the three expert agents critically examine one another's conclusions, assess the alignment between their own reasoning and that of others, and, in cases of disagreement, choose either to rebut or to revise their prior conclusions, ultimately converging on a collaborative inference grounded in multi-perspective.

²<https://bio.libretexts.org>

To assess the effectiveness of REASONVUL, we conducted extensive experiments on the PrimeVul [9] dataset, utilizing pairwise accuracy (PairAcc) [63] as a key evaluation metric. PairAcc is a strict metric that considers a vulnerability-fix pair correctly classified only if both the vulnerable code and its fixed version are accurately identified. Our evaluation compares REASONVUL against eight state-of-the-art baselines, including learning-based (LineVul [13], Coca [5], MoEVD [62]) and LLM-based (SAVul [6], Vul-RAG [10], VulnSage [76], GPTLens [20], and VulTrial [58]) methods. The results demonstrate that REASONVUL significantly outperforms all baselines, achieving a PairAcc of 40.00% and an F1-score of 72.52%, surpassing the best baseline (VulTrial) by 81.24% in PairAcc and 29.09% in F1-score. Furthermore, we conducted an in-depth analysis of the debate mechanism, revealing that it correctly resolves about 72% of conflicting cases (389 out of 542), compared to only about 9% with majority voting. These findings highlight REASONVUL's superior capability to detect complex vulnerabilities through cognitive diversity and structured debate.

In summary, our study makes the following contributions:

- **New Perspective.** We are the first to integrate deductive, inductive, and abductive reasoning within a multi-agent collaborative framework for vulnerability detection, thereby establishing a paradigm that more closely mirrors the cognitive processes of human experts.
- **Approach.** We design and implement REASONVUL, which leverages LLMs to assume the roles of security experts with distinct reasoning strengths, and employs a debate mechanism to drive their collaborative decision-making.
- **Experiment.** We conduct extensive experiments on the PrimeVul dataset, and the results demonstrate that REASONVUL significantly outperforms other state-of-the-art LLM-based approaches. The code and results are publicly available on our online repository.

2 Related Works

2.1 Learning-Based Vulnerability Detection Approaches

The general paradigm of these approaches is to train deep learning models on labeled code snippet datasets to automatically learn deep features and patterns of vulnerabilities [59, 67]. They are categorized into graph-based and token-based methods based on code representation [70]. Specifically, graph-based methods focus on capturing the structural information of code. They model code as graph structures and leverage graph neural networks (GNNs) to learn representations [37, 38, 68]. For instance, Devign [72] constructs a code property graph by integrating abstract syntax trees (AST), control flow graphs (CFG), and data flow graphs (DFG) to capture complex structural relationships, utilizing a gated graph neural network (GGNN) as an encoder to learn vulnerability-related features. While graph-based models excel at capturing explicit code structures, the emergence of Transformer architectures and large-scale pretraining has opened new avenues for understanding deep code semantics [69]. Consequently, token-based methods leverage pretrained models like CodeBERT to extract code semantics. For example, studies such as LineVul [13], Stagedvulbert [23], and Vulcobert [60] have adopted CodeBERT [12] as their core feature extractor, achieving rapid convergence and superior performance. To further enhance detection accuracy, the MoEVD [62] model introduces a Mixture-of-Experts framework, decomposing the broad vulnerability detection task into subtasks targeting specific CWEs, thereby enabling finer-grained classification and detection. Despite the advantages of learning-based methods, they also face certain limitations. First, their performance is highly dependent on the quality and diversity of training data, and their generalization ability is often limited when encountering vulnerabilities not present in the training set [34]. Second, these methods primarily focus on syntactic or structural patterns in code, making it challenging to identify vulnerabilities requiring contextual understanding at the semantic or logical level [42]. These shortcomings have spurred exploration into methods based on LLMs.

2.2 LLM-Based Vulnerability Detection Approaches

Unlike learning-based methods, LLMs leverage their natural language understanding and code comprehension capabilities to identify vulnerabilities without extensive task-specific fine-tuning. Preliminary explorations show that LLMs exhibit considerable potential in vulnerability detection even under zero-shot or few-shot settings [66]. Recent studies further reveal the deeper capabilities of LLMs. For instance, it has been found that when guided by structured prompting techniques such as CoT, LLMs can outperform pre-trained language models like CodeBERT that have been specifically fine-tuned [9, 15, 75]. This finding is significant as it suggests that the advanced reasoning abilities unlocked through prompting may be more effective for handling complex vulnerability analysis tasks than the pattern-recognition abilities gained through traditional supervised learning. To further enhance the depth of LLM reasoning, researchers have explored more sophisticated mechanisms inspired by human expert cognition [50, 71]. These include the adoption of self-reflection mechanisms, which prompt LLMs to critically review and revise their initial conclusions, as well as the use of RAG, which incorporates relevant historical vulnerability patterns from external knowledge bases into the reasoning process to improve accuracy [8].

Recently, researchers have constructed adversarial detection frameworks through role-playing [54, 57]. The core idea of such methods is to simulate expert teams engaging in mutual rebuttal to enhance detection performance. For example, VulTrial [58] adopts a courtroom-inspired design to build a more robust vulnerability detection framework, where LLMs play the roles of prosecutor, defender, and judge, followed by another LLM acting as a jury to make the final verdict on the presence of a vulnerability. Although these methods demonstrate the potential of collaborative reasoning, their role design is fundamentally based on role-playing. As their interactions largely amount to mutual questioning, the reasoning process approaches a form of extended self-reflection [21]. In contrast, REASONVUL proposes a fundamentally different approach. Instead of mimicking procedural roles, our agent framework designates each agent to embody a core cognitive reasoning mode (i.e., deduction, induction, or abduction). This design seeks to build a more comprehensive analysis process by integrating distinct yet complementary logical perspectives, thereby more faithfully replicating the multidimensional reasoning process of human experts when tackling complex problems.

3 Motivation

While recent LLM-based methods have shown considerable promise in vulnerability detection, their effectiveness is often constrained by an implicit reliance on a single reasoning paradigm. To investigate this limitation, we conducted an empirical study on the PairVul dataset [10], analyzing how representative state-of-the-art tools, each embodying a distinct reasoning mode, perform on real-world vulnerabilities. We selected SAVul [6] as an exemplar of deductive reasoning, Vul-RAG [10] for inductive reasoning, and VulnSage [76] for abductive reasoning. Our quantitative and qualitative analyses, detailed below, reveal that these paradigms exhibit unique strengths and critical blind spots, underscoring the need for an integrated approach.

We first evaluated the performance of SAVul, Vul-RAG, and VulnSage on the PairVul dataset and visualized the overlap of vulnerabilities they correctly

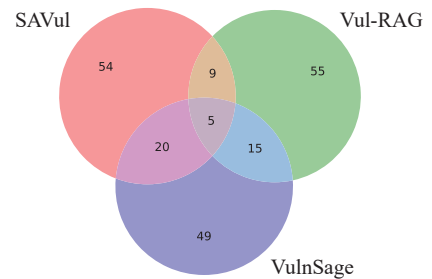


Fig. 1. The overlap of vulnerabilities correctly identified by SAVul (Deductive), Vul-RAG (Inductive), and VulnSage (Abductive) on the PairVul dataset.

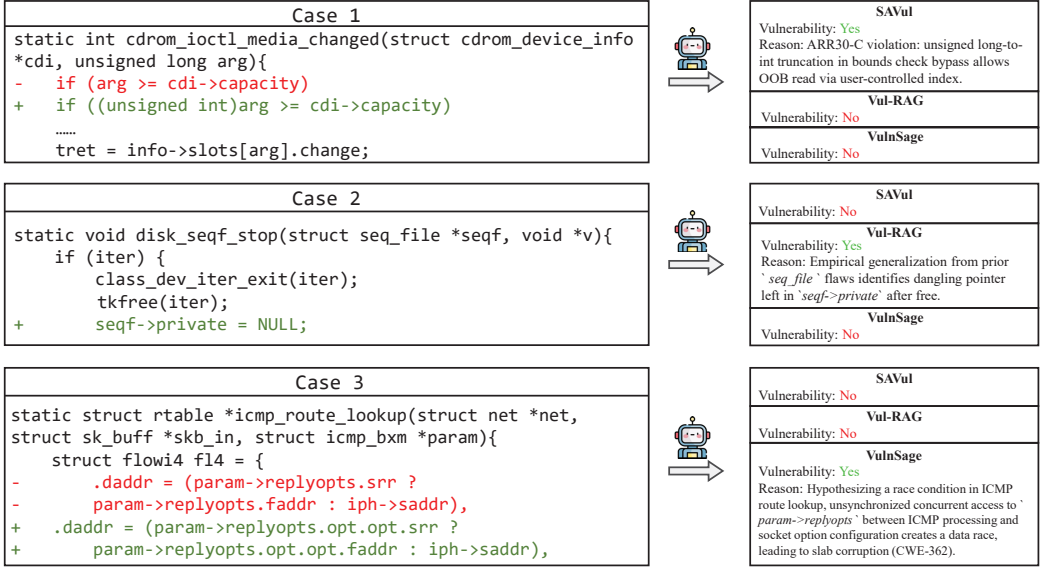


Fig. 2. Three motivating cases from the Linux kernel, selected from the PairVul dataset, demonstrating the distinct roles of deductive, inductive, and abductive reasoning in vulnerability detection.

identified. As shown in the Venn diagram in Fig. 1, the intersection among the sets of detected vulnerabilities is strikingly small. Each method identifies a large number of unique vulnerabilities that the other two miss, demonstrating that deductive, inductive, and abductive reasoning paradigms are highly complementary rather than redundant. This quantitative evidence confirms that relying on a single reasoning paradigm inherently leaves significant portions of vulnerabilities undetected due to paradigm-specific blind spots.

To further illustrate this complementarity, we selected three representative cases from the PairVul dataset, each highlighting the distinct strengths and limitations of the reasoning paradigms. These cases, drawn from the Linux kernel, demonstrate how deductive, inductive, and abductive reasoning excel at detecting different types of vulnerabilities.

Case 1: Integer Overflow. As illustrated in Fig. 2 (Case 1), an integer overflow vulnerability exists in the `cdrom_ioctl_media_changed` function. The code casts an `unsigned long` variable `arg` to an `unsigned int` before a bounds check. This explicit downcasting can cause a large input value for `arg` to wrap around to a small integer, thereby bypassing the `arg < cdi->capacity` check and leading to an out-of-bounds access at `info->slots[arg]`. Among the three approaches, only SAVul, leveraging deductive reasoning, successfully detected this vulnerability. Its top-down approach applied the general security principle that “type casting between integer types of different sizes can cause overflows” directly to the code, thereby flagging the `(unsigned int)arg` cast as a violation. In contrast, Vul-RAG, relying on inductive reasoning, failed to detect the issue due to its inability to match the code to known vulnerability patterns in its knowledge base, limiting its generalization capability. Similarly, VulnSage, using abductive reasoning, incorrectly assumed the issue was a simple out-of-bounds access. It traced back to the bounds check, deemed it sufficient, and missed the subtle integer overflow within the type cast, concluding no vulnerability existed.

Case 2: Use-After-Free. Case 2 involves a use-after-free vulnerability in the `disk_seqf_stop` function. The code frees the memory allocated to the `iter` pointer using `kfree(iter)` but neglects

to set the `seqf->private` pointer, which references the freed `iter`, to `NULL`. Subsequent calls to the function under specific conditions may dereference this dangling pointer, causing memory corruption or undefined behavior. This vulnerability was successfully detected only by Vul-RAG. Its pattern-based approach retrieved a similar vulnerability pattern related to the `seq_file` function from its knowledge base, identifying the dangling pointer in `seqf->private` as a match. However, SAVul and VulnSage failed to detect the issue. SAVul could not link the missing nullification to a specific security rule, while VulnSage's hypothesized attack scenario did not account for the pointer impact, missing the vulnerability entirely.

Case 3: Race Condition. The third case, selected from the PairVul dataset, involves a complex race condition in the Internet Control Message Protocol (ICMP) routing logic, which handles network communication tasks such as error reporting and diagnostics in the Linux kernel. Improper access to a shared structure under specific network traffic conditions can cause slab corruption and system crashes. Both SAVul and Vul-RAG, relying on deductive and inductive reasoning, respectively, misclassified the vulnerability as non-existent. SAVul struggles to extract general security principles suitable for deductive application from this code, resulting in the race condition being overlooked, and Vul-RAG could not generalize from its knowledge base due to the rarity of similar cases. In contrast, VulnSage, using abductive reasoning, successfully detected the vulnerability. By hypothesizing a race condition in the ICMP route lookup, it traced back to unsynchronized access to `param->replyopts` between ICMP processing and socket option configuration, correctly identifying a data race leading to slab corruption (CWE-362).

These case studies confirm that deductive reasoning excels at detecting rule-based vulnerabilities, inductive reasoning is effective for pattern-based, common vulnerabilities, and abductive reasoning is suited for uncovering complex, context-dependent vulnerabilities. Our findings reveal a fundamental limitation in LLM-based vulnerability detection: reliance on a single mode of reasoning. Consequently, merely adopting any one of these paradigms is insufficient to effectively address the diverse and complex nature of real-world vulnerabilities. Motivated by this insight, we propose REASONVUL, a collaborative framework designed to harness the power of **multi-perspective reasoning** through a process of **cognitive synergy**. The core idea of REASONVUL is to first allow agents specializing in deductive, inductive, and abductive reasoning to independently analyze the code, maximizing the breadth of initial findings. Subsequently, it initiates a structured debate mechanism where agents cross-validate, challenge, and synthesize their findings, aiming to converge on a final judgment with significantly higher accuracy and reliability.

4 Approach

4.1 Overview

REASONVUL leverages an LLM-based multi-agent system to achieve accurate vulnerability detection through cognitive synergy. The core idea is to simulate a team of security experts with diverse reasoning skills who collaborate to reach a reliable consensus. As illustrated in Fig. 3, REASONVUL operates in two sequential stages:

- **Multi-perspective Reasoning:** Three specialist agents, each embodying a distinct reasoning paradigm: deductive, inductive, or abductive, independently analyze the input source code. This parallel analysis ensures a broad and diverse set of initial findings, maximizing the chances of uncovering different types of vulnerabilities.
- **Collaborative Debate:** If the agents' initial judgments conflict, they enter a multi-round debate. In this stage, they exchange, challenge, and defend their reasoning, iteratively refining their positions to resolve discrepancies and synthesize their unique insights.

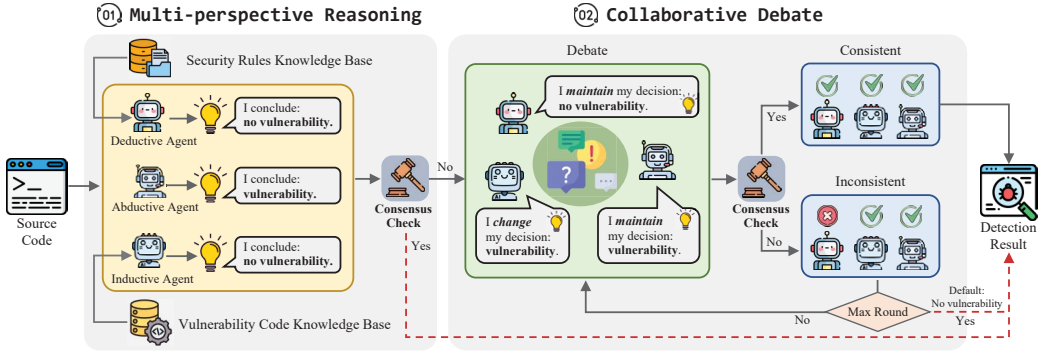


Fig. 3. The workflow of REASONVUL.

This two-stage architecture is a direct response to our findings in Section 3. Relying on a single reasoning paradigm results in critical blind spots, while simplistic aggregation methods like majority voting are insufficient for resolving nuanced disagreements. By contrast, REASONVUL’s debate mechanism facilitates a deeper integration of perspectives, leading to more accurate and robust vulnerability detection.

4.2 Multi-perspective Reasoning

The Multi-perspective Reasoning stage lays the groundwork for REASONVUL’s collaborative analysis. Its primary objective is to generate a diverse set of independent assessments of a given source code snippet X . Each of the three agents, $\mathcal{A}_i$ (where $i \in \{D, I, A\}$ for Deductive, Inductive, and Abductive), is designed to operate in isolation, preventing premature convergence and ensuring a wide range of potential issues are considered.

To precisely define their functions, we formally model each agent’s operation. An agent $\mathcal{A}_i$ acts as a function f_i that takes the source code X and a paradigm-specific knowledge source K_i as input. It produces an output tuple $O_i = (v_i, e_i)$, where $v_i \in \{0, 1\}$ is the binary vulnerability judgment (1 for vulnerable, 0 for benign) and e_i is a natural language explanation detailing its reasoning. This process is defined as $O_i = f_i(X, K_i)$. This formalization clarifies the distinct inputs and consistent output structure for each agent before we delve into their unique internal processes. Detailed information regarding the specific prompts and construction process for all agents can be found in the Data Availability section 9, which provides the online repository.

4.2.1 Deductive Agent. The Deductive Agent emulates a security auditor who applies general principles to specific instances. It operates top-down, using established secure coding standards to identify violations. The rationale is that many vulnerabilities, such as integer overflows or improper resource handling, are direct violations of well-defined rules.

Implementation. The agent’s analysis is powered by a Retrieval-Augmented Generation (RAG) mechanism designed to ground its reasoning in authoritative security knowledge. Its knowledge base, K_D , is derived from the SEI CERT C Coding Standard.³ We structure it as a collection of pairs, $K_D = \{(\text{desc}_k, \text{rule}_k)\}_{k=1}^M$, where each desc_k is the natural language description of a potentially flawed behavior (e.g., "Accessing freed memory corrupts heap data structures."), and rule_k is the corresponding formal coding rule (e.g., "MEM30-C. Do not access freed memory").

³<https://wiki.sei.cmu.edu>

During an offline indexing phase, we encode all natural language descriptions desc_k into high-dimensional vectors using the Jina-V3 embedding model [51], denoted as $\mathcal{E}(\cdot)$, creating an efficient retrieval index. When a source code snippet X is provided for analysis, it is also embedded into a vector $\mathcal{E}(X)$. The system then computes a similarity score $s_k = \text{cos_sim}(\mathcal{E}(X), \mathcal{E}(\text{desc}_k))$ for every description in the knowledge base. The indices of the top-5 descriptions with the highest scores, denoted by the set $\mathcal{J}^*$, are identified.

The corresponding set of rules, $\{\text{rule}_k \mid k \in \mathcal{J}^*\}$, is then retrieved from the knowledge base. These rules are integrated into the final prompt, P_D , along with the input code X . The prompt's structure, guided by our template, instructs the Deductive Agent to act as a meticulous security auditor. It must systematically check if the code X violates any of the provided high-relevance security rules and explain its findings. Finally, this context-rich prompt P_D is processed by the agent's underlying LLM to produce the judgment and explanation $O_D = (v_D, e_D)$, ensuring the analysis is firmly grounded in established secure coding standards.

4.2.2 Inductive Agent. The Inductive Agent mirrors a data-driven security practitioner who generalizes from historical data to identify risks. It operates via a bottom-up, pattern-matching approach. The core intuition is that many software vulnerabilities are not entirely novel but are recurrences of previously discovered flawed patterns. By retrieving a known, analogous vulnerability instance, the agent can infer the risk in the new code snippet, much like an expert recalling a past incident to diagnose a present problem.

Implementation. The agent's reasoning is grounded in a knowledge base of real-world security issues. Following prior research [29], we use the **ReposVul** dataset [53]. This provides a knowledge base $K_I = \{(c_{\text{vuln}}^{(j)}, c_{\text{fix}}^{(j)})\}_{j=1}^N$ containing $N = 6,956$ function-level vulnerability-fix pairs. Crucially, to ensure a fair evaluation and prevent data leakage, we meticulously filter this dataset, removing any examples that also appear in our downstream evaluation benchmarks.

The core of the agent's implementation is an RAG pipeline that retrieves the most relevant historical example. During an offline phase, every vulnerable function $c_{\text{vuln}}^{(j)} \in K_I$ is encoded into a high-dimensional vector using the Jina-V3 embedding model, which we denote as a function $\mathcal{E}(\cdot)$. When a new source code snippet X is presented for analysis, it is also embedded into a vector $\mathcal{E}(X)$. The system then identifies the index j^* of the most similar historical case by solving: $j^* = \arg \max_j \text{cos_sim}(\mathcal{E}(X), \mathcal{E}(c_{\text{vuln}}^{(j)}))$. The corresponding pair, $(c_{\text{vuln}}^{(j^*)}, c_{\text{fix}}^{(j^*)})$, is then retrieved from the knowledge base to serve as a potent in-context example. The final prompt, P_I , is then carefully constructed. It incorporates the retrieved pair $(c_{\text{vuln}}^{(j^*)}, c_{\text{fix}}^{(j^*)})$ as a concrete example alongside the input code X that requires analysis. This entire context is framed by a set of predefined instructions from our prompt template, which guides the agent to first analyze the historical example to understand the vulnerability and its fix, and second, to determine if the new code snippet exhibits a similar flawed pattern. This prompt P_I is then processed by the agent's underlying LLM to generate the final judgment and explanation $O_I = (v_I, e_I)$.

4.2.3 Abductive Agent. The Abductive Agent is designed to emulate the creative and exploratory mindset of a penetration tester. It reasons backward, starting from a hypothesized adverse outcome (an effect) and working to find the most plausible vulnerability in the code (the cause). This hypothesis-driven approach is uniquely suited for uncovering complex, context-dependent flaws such as race conditions, intricate logic bugs, or novel attack vectors that are not captured by predefined rules or historical patterns.

Implementation. The Abductive agent is centered on a sophisticated multi-step reasoning prompt that guides the LLM through a structured, internal thought process. Unlike its counterparts, $\mathcal{A}_A$ does not utilize an external knowledge base via RAG. Instead, it leverages the LLM's intrinsic,

Table 1. The PairAcc (%) of different LLMs on the PairVul dataset using three reasoning paradigms.

Paradigm	Open Source			Commercial			
	Llama4: 17b	Phi4: 14b	Devstral: 24b	GPT-4o	DeepSeek	Qwen3	Gemini 2.5
Deductive	15.17	21.89	18.41	18.41	19.15	19.70	14.43
Inductive	20.90	18.41	12.94	12.19	18.41	16.17	10.95
Abductive	17.41	17.41	13.43	11.19	22.14	15.42	6.72

pre-trained knowledge of software, security, and attack patterns as its internal knowledge source, K_A . This allows it to generate novel hypotheses beyond the confines of any specific dataset.

The agent's prompt, P_A , instructs it to adopt the persona of an attacker and perform a structured CoT analysis on the input code X . The process is as follows: first, the agent must identify potentially risky code segments and generate a set of plausible attack hypotheses, $H = \{h_1, \dots, h_n\}$, regarding how the code might fail or be exploited. For each leading hypothesis $h \in H$, it is then required to construct a detailed reasoning trace, $T(h)$, outlining a potential attack path and explaining how specific lines of code contribute to the vulnerability. Finally, the agent must perform a crucial self-critique step, where it re-evaluates its own reasoning trace to assess its logical soundness and the real-world plausibility of the exploit.

The final explanation, e_A , is a coherent synthesis of this entire internal monologue that goes from initial hypothesis to the validated conclusion. This structured, self-correcting reasoning process, encapsulated within the prompt P_A , enables the agent to form and validate complex arguments, ultimately leading to its final judgment $O_A = (v_A, e_A)$.

4.2.4 Model Specialization. Recognizing that a one-size-fits-all approach is suboptimal, and that LLM architecture and training data lead to varying aptitudes for different reasoning tasks [50, 64], we conducted an empirical study to assign the best-performing model to each reasoning paradigm. Using the PairVul dataset [10], we evaluated model performance with PairAcc. Our experimental results under this metric, summarized in Table 1, revealed clear specializations. Consequently, we configured our agents to leverage these model-specific strengths: the Deductive Agent is powered by Phi4 (21.89% PairAcc), the Inductive Agent employs Llama4 (20.90%), and the Abductive Agent leverages DeepSeek (22.14%). This data-driven assignment ensures that each agent operates at its peak empirical performance, thereby maximizing the quality and diversity of the initial judgments before the collaborative debate begins.

4.2.5 Phase Conclusion and Transition. The Multi-perspective Reasoning phase concludes once all three specialized agents have produced their independent outputs. These results are aggregated into a collective set, $\mathcal{O}^0 = \{O_D^0, O_I^0, O_A^0\}$. The framework then determines the next step based on a transition variable, $\mathcal{T}^0$, which checks for consensus:

$$\mathcal{T}^0 = \begin{cases} \text{exit,} & \text{if } v_D^0 = v_I^0 = v_A^0 \\ \text{debate,} & \text{otherwise} \end{cases}$$

If all agents are in agreement ($\mathcal{T}^0 = \text{exit}$), their consistent judgment $v_f = v_D^0$ is accepted as the final result, and their individual explanations $\{e_D^0, e_I^0, e_A^0\}$ are synthesized into a single, coherent report. However, if there is any disagreement ($\mathcal{T}^0 = \text{debate}$), the framework advances to the Collaborative Debate phase to resolve the conflict.

4.3 Collaborative Debate

While the Multi-perspective Reasoning phase generates a crucial diversity of initial opinions, these independent analyses are often insufficient to resolve complex cases where paradigms conflict.

Relying on simplistic aggregation methods, such as majority voting, risks discarding the nuanced insights of a dissenting agent. This phase realizes our key insight that cognitive synergy is achieved through iterative interaction. We introduce the Collaborative Debate, a structured process designed to simulate the scientific principle that truth emerges from critical discourse [32]. By compelling agents to confront, justify, and synthesize differing viewpoints, the debate forces a re-examination of evidence that may have been initially overlooked or misinterpreted. This iterative refinement is the engine that drives the agents toward a unified consensus that is more robust and reliable than any of their isolated, initial judgments.

The debate commences if the initial judgments diverge and proceeds in rounds, $t = 1, 2, \dots, T_{\max}$. A core design principle is that agents deliberate and update their views in parallel within each round. This “simultaneous speaking” model prevents sequential bias, where the order of speakers could unduly influence the outcome, and ensures each reasoning paradigm is given equal weight [11]. In any given round t , each agent $\mathcal{A}_i$ receives a comprehensive context for its deliberation. This context includes the source code X , its specific knowledge K_i , its own reasoning history $\{O_i^0, \dots, O_i^{t-1}\}$, and the most recent outputs from its peers, $\{O_j^{t-1} \mid j \neq i\}$.

Within each round, every agent performs a parallel deliberation process. It first critically audits the reasoning chains (e_j^{t-1}) of its peers, especially those with conflicting judgments ($v_j^{t-1} \neq v_i^{t-1}$). It then re-evaluates the evidence through the lens of its own paradigm. If a peer’s argument is persuasive and reveals a flaw in its own prior logic, the agent revises its stance. Conversely, if it finds the peer’s reasoning unsound, it formulates a rebuttal, reinforcing its original position and attempting to persuade others. This deliberation process, which we denote as the function g_i , generates an updated output tuple for the agent: $O_i^t = (v_i^t, e_i^t) = g_i(X, K_i, \{O_i^0, \dots, O_i^{t-1}\}, \{O_j^{t-1} \mid j \neq i\})$.

After each round of deliberation, the framework aggregates the new outputs into a set $\mathcal{O}^t = \{O_D^t, O_I^t, O_A^t\}$ and checks for consensus by evaluating the transition state $\mathcal{T}^t$:

$$\mathcal{T}^t = \begin{cases} \text{exit}, & \text{if } v_D^t = v_I^t = v_A^t \\ \text{debate}, & \text{otherwise} \end{cases}$$

If consensus is reached ($\mathcal{T}^t = \text{exit}$), the debate terminates. The unanimous decision becomes the final judgment, v_f , and a final explanation, e_f , is synthesized from the agents’ convergent reasoning chains. If no consensus is reached after $T_{\max}$ rounds, the framework defaults to a non-vulnerable judgment ($v_f = 0$). This is a deliberate design choice, motivated by empirical findings that LLMs are prone to false positives in vulnerability detection [58, 75] and aligns with the security principle of minimizing alert fatigue.

5 Experiment Design

5.1 Research Questions

To evaluate the effectiveness of our approach, we address the following research questions:

- **RQ1:** How does REASONVUL compare to state-of-the-art vulnerability detection approaches?
- **RQ2:** How is the generalizability of REASONVUL in software vulnerability detection?
- **RQ3:** What are the contributions of the major components of REASONVUL?

5.2 Dataset

We evaluate REASONVUL on the PrimeVul [9] dataset, a high-quality function-level benchmark for vulnerability detection that addresses label inaccuracies through combined automated labeling and manual verification, coupled with rigorous de-duplication. PrimeVul features diverse vulnerabilities across 140 CWE categories, sourced from large-scale open-source projects, with a test set of 435 vulnerable/fixed code pairs. To further validate generalization (RQ2), we utilize the JITVUL [63]

dataset, a repository-level benchmark derived from real-world projects, providing interprocedural context via caller-callee relationships. JITVUL includes 1758 paired samples, comprising 879 vulnerable and 879 benign versions, spanning 91 CWEs from 879 CVEs.

5.2.1 Baselines. We select baselines from two major categories, covering representative state-of-the-art techniques in vulnerability detection: learning-based methods and LLM-based methods. The learning-based baselines include CausalVul [40], DeepDFA [48], VulChecker [33], FVD-DPM [44], Coca [5], VulSim [47], LineVul [13], and MoEVD [62]. The LLM-based baselines include SAVul [6], Vul-RAG [10], VulnSage [76], GPTLens [20], and VulTrial [58]. To ensure a fair comparison, all baseline methods were evaluated under the same experimental environment and were re-executed using their officially provided publicly available implementations.

5.3 Evaluation Metrics

To comprehensively evaluate the performance of REASONVUL in vulnerability detection, we employ a suite of metrics, with a primary focus on PairAcc, as it is currently the most prominent and representative evaluation metric in the vulnerability detection domain [9, 58, 63]. PairAcc imposes a dual requirement that necessitates the correct classification of both a vulnerable function and its fixed version. This metric essentially serves as a strict standard that balances the need for high Recall with the necessity of a low False Positive Rate. In practical scenarios, suppressing false positives is crucial to maintaining developer trust and reducing alert fatigue. By requiring the model to distinguish between the vulnerable and fixed code, PairAcc effectively penalizes models that tend to aggressively predict vulnerabilities, ensuring that the detector possesses the discriminative capability required to minimize false alarms on benign code. For each (vulnerable, fixed) pair, the model must predict the former as “1” and the latter as “0”. Formally, PairAcc is defined as:

$$\text{PairAcc} = \frac{|\{(v, f) \in \mathcal{P} \mid \hat{y}_v = 1 \wedge \hat{y}_f = 0\}|}{|\mathcal{P}|}, \quad (1)$$

where $\mathcal{P}$ represents the set of all (vulnerable, fixed) code pairs, (v, f) denotes a single pair with v as the vulnerable code and f as the fixed code, $\hat{y}_v$ and $\hat{y}_f$ are the predictions for the vulnerable and fixed code snippets, respectively, and $|\mathcal{P}|$ is the total number of pairs. In addition to PairAcc, we report widely-used classification metrics: **Accuracy**: Accuracy measures the overall correctness of predictions. **Precision**: Precision measures the proportion of true vulnerabilities among all predicted vulnerable instances. **Recall**: Recall measures the proportion of actual vulnerabilities correctly identified. **F1-score**: F1-score measures the harmonic mean of precision and recall. **False Positive Rate (FPR)**: FPR measures the proportion of non-vulnerable code incorrectly predicted as vulnerable.

5.4 Implementation Details

For open-source LLMs, we implement REASONVUL on a server equipped with four NVIDIA RTX 4090 GPUs. We download the model weights of Llama-4⁴, Phi-4⁵, and Devstral⁶ from the HuggingFace and deploy them locally. For commercial LLMs, we access the models directly through their official APIs. Specifically, we employ the following model versions: gpt-4o-2024-05-13, qwen-max, gemini-2.5-flash, and deepseek-v3-0324. Regarding the generation configuration, we set the temperature to 0 to minimize randomness and ensure the reproducibility of our results. For all other hyperparameters, we adopt the default settings provided by the openai library⁷: top_p is

⁴<https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct>

⁵<https://huggingface.co/microsoft/phi-4>

⁶<https://huggingface.co/mistralai/Devstral-Small-2505>

⁷<https://github.com/openai/openai-python>

Table 2. Comparison with state-of-the-art vulnerability detection techniques on the PrimeVul Dataset

Type	Method	PairAcc ($\uparrow$)	Accuracy ($\uparrow$)	Precision ($\uparrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)	FPR ($\downarrow$)
Learning-based	CausalVul	5.74	49.43	49.44	51.03	50.23	52.18
	DeepDFA	4.36	49.54	49.51	46.21	47.80	47.12
	VulChecker	3.67	48.28	47.81	37.70	42.16	41.15
	FVD-DPM	5.52	52.30	53.28	44.83	48.69	39.31
	Coca	4.14	50.92	50.91	51.95	51.42	50.11
	VulSim	8.51	49.20	49.28	54.71	51.85	56.32
	LineVul	4.36	52.07	52.31	46.90	49.45	42.76
	MoEVD	8.97	53.67	54.12	48.27	51.03	40.92
LLM-based	SAVul	11.03	53.91	55.28	40.92	47.28	37.93
	Vul-RAG	10.57	51.95	52.00	51.03	51.51	47.13
	VulnSage	8.28	48.97	48.31	29.66	36.75	31.72
	GPTLens	13.56	51.73	51.20	78.16	61.87	74.71
	VulTrial	22.07	55.17	54.95	57.47	56.18	47.13
LLM-based	REASONVUL	40.00	69.77	66.48	79.77	72.52	40.23

set to 1.0, top_k is set to 50, and repetition_penalty is set to 1.0. For the parameter settings of REASONVUL, we explored the maximum number of debate rounds $T_{\max}$ within a selection space of 1 to 5. Based on subsequent ablation experiments, we ultimately set $T_{\max}$ to 2.

6 Experiment Results

6.1 RQ1: Compare with State-of-the-Art Approaches

To answer RQ1, we compare ReasonVul with a comprehensive set of state-of-the-art baseline methods, including both learning-based and LLM-based approaches. For learning-based methods, we train each model on the PrimeVul training set using the official hyperparameter configurations provided by the paper. For models with limited context window capacity, inputs that exceed the maximum supported context length are truncated following the constraints of the original implementations. For example, CausalVul, LineVul, and MoEVD are implemented based on CodeBERT, which supports a maximum input length of 512 tokens. Note that we adapted the line-level tool (i.e., VulChecker) to the function-level detection task to ensure consistency of metrics across methods. The trained models are then evaluated on the PrimeVul test set using the learned weights. For LLM-based methods, we directly apply their officially released prompts and pipelines to the PrimeVul test set without additional fine-tuning. Finally, we report PairAcc, accuracy, precision, recall, F1-score, and FPR for all methods.

Table 2 summarizes the performance of REASONVUL and the baseline methods on the PrimeVul dataset. Overall, our experimental results demonstrate that REASONVUL achieves superior performance across all evaluation metrics. Specifically, REASONVUL attains a PairAcc of 40.00%, indicating its exceptional ability to accurately distinguish between vulnerable and fixed code pairs, capturing subtle semantic differences that characterize real-world vulnerabilities. This result represents substantial improvements over the best-performing baseline (VulTrial), with gains of 81.24% in PairAcc. This highlights the effectiveness of our multi-agent debate framework in leveraging diverse reasoning paradigms for more accurate and reliable vulnerability detection.

Learning-based methods exhibit moderate performance, with PairAcc values ranging from 3.67% to 8.97% and F1-scores around 42%-52%. These approaches, reliant on supervised learning from labeled datasets, are limited by their focus on syntactic and structural patterns, which hinders their ability to distinguish subtle semantic differences between vulnerable and fixed code pairs. Their low

PairAcc values reflect this limitation, as they struggle to generalize to unseen vulnerabilities or capture the nuanced characteristics required for accurate pairwise detection. The LLM-based methods generally demonstrate better results than the learning-based methods, due to their powerful code understanding capabilities, with PairAcc ranging from 8.28%-10.57%. Specifically, VulnSage, which leverages an abductive reasoning approach through a self-reflection and validation mechanism, tends to output benign results, resulting in a lower recall rate. Vul-RAG, employing an inductive reasoning strategy with RAG, improves performance by increasing the recall rate to 51.03% and achieving an F1-score of 51.51%. However, these methods are still limited by the inherent reasoning capabilities of a single LLM, and their performance remains constrained when dealing with complex code semantics. The GPTLens and VulTrial outperform the previous approaches, with PairAcc values of 13.56% and 22.07%, respectively. Notably, while GPTLens achieves a high recall rate of 78.16%, it suffers from a significantly high FPR of 74.71%, indicating a severe tendency for over-detection. In contrast, VulTrial introduces roles for a judge and a jury to simulate a courtroom debate mechanism, improving the balance of performance. However, REASONVUL achieves a much lower FPR of 40.23% while maintaining high recall, demonstrating superior capability in reducing false alarms compared to adversarial baselines. Therefore, REASONVUL demonstrates superior vulnerability detection capability compared to the baseline methods.

Table 3. Accuracy Comparison on Top-10 Most Frequent CWEs in PrimeVul.

CWE ID	Vulnerability Type	Count	GPTLens	VulTrial	REASONVUL
CWE-787	Out-of-bounds Write	144	53.47	54.86	66.67
CWE-125	Out-of-bounds Read	94	53.19	54.26	75.53
CWE-703	Improper Check/Handling of Exceptions	94	56.38	51.06	72.04
CWE-476	NULL Pointer Dereference	78	48.72	53.85	75.64
CWE-416	Use After Free	58	41.38	53.45	68.97
CWE-200	Exposure of Sensitive Information	32	53.12	62.50	59.38
CWE-20	Improper Input Validation	28	53.57	50.00	78.57
CWE-369	Divide By Zero	28	57.14	46.43	67.86
CWE-119	Improper Restriction of Operations	28	53.57	57.14	71.43
CWE-617	Reachable Assertion	24	62.50	54.17	70.83
Average			53.30	53.77	70.69

Performance across Different Vulnerability Types. To assess the efficacy of ReasonVul in addressing real-world security threats, we conducted a thorough investigation into the Top-10 most frequent CWEs in the PrimeVul dataset. These vulnerabilities represent the most common occurrences in our dataset and pose significant risks due to their inherent ease of discovery and exploitation. We compare the accuracy of ReasonVul against two LLM-based baselines, GPTLens and VulTrial.

As shown in Table 3, REASONVUL consistently outperforms the baselines across nearly all frequent CWE categories, achieving an average accuracy of 70.69%, which is substantially higher than GPTLens at 53.30% and VulTrial at 53.77%. This advantage indicates that REASONVUL is not merely improving the aggregate performance reported in Table 2, but also provides more stable detection capability when the vulnerabilities are grouped by their underlying weakness types. Overall, these results verify that the cognitive synergy of deductive, inductive, and abductive reasoning effectively addresses diverse vulnerability characteristics, offering superior robustness over role-playing approaches.

Table 4. Performance comparison of repository-level vulnerability detection on the JITVUL dataset

Type	Method	PairAcc (↑)	Accuracy (↑)	Precision (↑)	Recall (↑)	F1-score (↑)	FPR (↓)
Learning-based	CausalVul	3.07	51.54	51.50	52.90	52.19	49.83
	DeepDFA	3.98	51.99	52.25	46.19	49.03	42.21
	VulChecker	3.19	51.59	51.83	45.05	48.20	41.87
	FVD-DPM	4.32	52.16	52.11	53.36	52.73	49.03
	Coca	2.05	50.85	50.49	87.71	64.09	86.01
	VulSim	6.94	53.47	53.17	58.25	55.59	51.31
	LineVul	5.12	51.65	51.75	48.69	50.18	45.39
	MoEVD	5.80	52.28	51.85	63.82	57.22	59.27
LLM-based	SAVul	8.99	49.77	49.84	68.83	57.81	69.28
	Vul-RAG	12.97	54.32	53.51	65.87	59.05	57.22
	VulnSage	9.22	50.17	50.27	32.08	39.17	31.74
	GPTLens	10.69	52.90	51.68	89.42	65.50	82.13
	VulTrial	20.93	56.83	56.93	56.09	56.50	42.43
LLM-based	REASONVUL	28.67	63.54	61.85	70.65	65.96	43.57

Answer to RQ1: REASONVUL significantly outperforms all baseline methods in vulnerability detection, achieving the highest scores across all metrics. Compared to previous LLM-based multi-agent approaches, PairAcc improves by 81.24%-194.99%.

6.2 RQ2: Generalizability of REASONVUL

We have demonstrated that REASONVUL achieves impressive performance, successfully detecting vulnerabilities from the widely used PrimeVul dataset. To assess the generalizability of REASONVUL, we extend our evaluation from the function-level PrimeVul dataset to the more challenging repository-level JITVUL dataset. While PrimeVul provides clean, function-level code snippets, JITVUL introduces a higher degree of complexity by providing repository-level context, including interprocedural dependencies like caller-callee relationships. This setup more closely mirrors real-world code auditing scenarios where vulnerabilities often arise from interactions between different parts of a codebase (i.e., interprocedural vulnerabilities). In this RQ, we investigate whether REASONVUL can effectively detect interprocedural vulnerabilities given a suspicious function and its immediate calling context (i.e., its caller and callee functions).

To capture interprocedural dependencies, we employ CFlow [16] to statically analyze the source code and extract the caller and callee functions associated with each target function. Following the setting of prior works [56, 63], when multiple callers or callees exist, we retrieve the top-5 most similar callers and the top-5 most similar callees, and serialize them into a unified input sequence in the format of *[Caller Context] + [Target Function] + [Callee Context]*. The contextual information for each caller and callee includes both its function signature and function body. This structured input ensures that REASONVUL receives the necessary interprocedural contextual information to reason across procedure boundaries. To ensure the fairness of the experiment, we follow the same experimental setup as in RQ1, using identical baselines. In particular, for learning-based methods, we directly reuse the model weights trained on the PrimeVul training set when evaluating on the JITVUL dataset.

As shown in Table 4, REASONVUL achieves the highest performance across all metrics, with a PairAcc of 28.67% and an F1-score of 65.96%. Although all methods experience a performance drop compared to PrimeVul due to JITVUL's increased complexity, REASONVUL maintains a significant

advantage over prior approaches. Specifically, compared with VulSim, which achieves the best performance among learning-based methods, REASONVUL attains an absolute improvement of 21.73% in PairAcc and 10.37% in F1-score. Compared with VulTrial, which achieves the best performance among LLM-based methods, REASONVUL achieves an absolute improvement of 7.74% in PairAcc and 9.46% in F1-score. Notably, learning-based methods exhibit a noticeable performance degradation compared to their evaluation results in RQ1. Specifically, token-based methods constrained by context length limits (e.g., 512 tokens), truncate critical inter-procedural information [23], while graph-based methods fail when handling large-scale code graphs [55].

Among LLM-based methods, we observe that the F1-score improvement of REASONVUL over GPTLens is only 0.46%. However, a comprehensive analysis across all evaluation metrics reveals significant differences in detection capabilities. GPTLens primarily achieves high recall through aggressive predictions, which results in an extremely high FPR of 82.13% and leads to an inflated performance under aggregate metrics. In contrast, REASONVUL attains a higher F1-score while reducing the FPR by 38.56%. These results indicate that, compared to GPTLens, REASONVUL exhibits a stronger capability to detect vulnerabilities that require reasoning over interprocedural context.

Answer to RQ2: REASONVUL exhibits strong generalizability on the JITVUL dataset, significantly outperforming baselines in repository-level vulnerability detection, with improvements of 65.91% in PairAcc over the best baseline VulTrial.

6.3 Ablation Study

To dissect the contributions of REASONVUL's core components, we conduct ablation studies on the PrimeVul dataset, focusing on two key aspects: the impact of individual reasoning agents and the influence of debate rounds.

6.3.1 The Impact of Each Reasoning Thought. We first evaluate the contribution of each reasoning agent by comparing the full REASONVUL framework against configurations that either exclude one agent or use a single agent in isolation. Table 5 reports the performance metrics of different variants on the PrimeVul dataset. The experimental results show that the full REASONVUL configuration, which incorporates all three agents, achieves the highest performance across all metrics, underscoring the critical role of cognitive diversity in vulnerability detection. Removing any single agent results in a consistent performance decline, demonstrating that each agent contributes uniquely to the framework's effectiveness. Specifically, the impact of disabling the deductive agent is the smallest, resulting in a PairAcc of 35.00% and an F1-score of 66.67%, representing decreases of 5.00% and 5.85% compared to the full model. Going further, removing the inductive agent reduces PairAcc to 31.95% and F1-score to 66.45%, corresponding to drops of 8.05% and 6.07%. Finally, the most significant performance degradation occurs when the abductive agent is removed, with PairAcc falling to 24.60% and F1-score to 59.78%, decreases of 15.40% and 12.74%, respectively. This underscores the Abductive Agent's unique ability to identify complex, context-dependent vulnerabilities that other paradigms miss. Additionally, we observed that all single-agent configurations performed significantly worse than the full model, with PairAcc dropping by 12.91%–19.08% and F1-score decreasing by 10.38%–12.55%. This confirms that the integration of deductive, inductive, and abductive reasoning is essential for effectively capturing the diversity of vulnerabilities.

6.3.2 The Impact of Debate. We investigate the impact of varying the maximum number of debate rounds $T_{\max}$ from 0 to 5 on the PrimeVul validation set, where $T_{\max} = 0$ corresponds to relying solely on the multi-perspective reasoning phase with majority voting to resolve conflicts. The results, illustrated in Fig. 4, reveal performance trends for PairAcc, Accuracy, and F1-score across these

Table 5. The impact of individual reasoning agents

Configuration	PairAcc	Accuracy	Precision	Recall	F1-score
w/o Deductive Agent	35.00	64.45	68.09	65.31	66.67
w/o Inductive Agent	31.95	65.17	64.10	68.97	66.45
w/o Abductive Agent	24.60	57.47	56.70	63.22	59.78
Deductive Agent Only	25.28	58.16	57.17	65.06	60.86
Inductive Agent Only	20.92	55.06	67.36	67.36	59.97
Abductive Agent Only	27.81	62.18	62.21	62.07	62.14
Full	40.00	69.77	66.48	79.77	72.52

settings. At 0 rounds, PairAcc is 21.67%, Accuracy is 54.69%, and F1-score is 57.23%, highlighting the limitations of majority voting, which fails to exploit nuanced insights embedded in conflicting agent judgments.

Introducing debate substantially improves performance: with just 1 round, PairAcc increases to 26.46%, Accuracy to 59.38%, and F1-score to 61.54%. The best results are achieved at 2 rounds, where PairAcc reaches 30.42%, Accuracy 62.50%, and F1-score 63.41%, demonstrating that structured debate effectively promotes cognitive synergy through iterative refinement. Beyond 2 rounds, performance begins to decline. This decline is attributed to the excessively long context accumulated in multi-round dialogues, which can overwhelm the ability of agents to maintain coherent reasoning, leading to potential misinterpretations or conservative judgments [24]. These results validate our choice of $T_{\max} = 2$, which optimizes detection accuracy while maintaining computational efficiency.

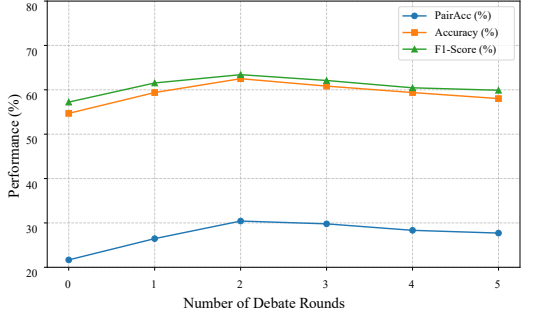


Fig. 4. The impact of debate rounds

In addition, to validate our design choice of defaulting to a benign judgment when consensus is not reached after $T_{\max}$ rounds, we conducted experiments on the test set that showed an absolute improvement of 4.14% in PairAcc compared to the majority voting strategy. These results indicate that adopting a conservative strategy after debate reduces false positives, which aligns with prior findings on the over-detection tendencies of LLMs [75].

Answer to RQ3: Each reasoning agent contributes uniquely, with their combination yielding substantial gains. The debate mechanism is crucial, with 2 rounds optimizing performance by enabling effective cognitive synergy.

7 Discussion

7.1 Case Study

To illustrate the critical role of the collaborative debate mechanism in REASONVUL, we present a case study analyzing a real-world integer overflow vulnerability from the Linux kernel⁸. This case is selected to demonstrate how integrating multiple reasoning paradigms, combined with

⁸<https://nvd.nist.gov/vuln/detail/CVE-2021-41864>

<pre>static int prealloc_elems_and_freelist(struct bpf_stack_map *smap){ - u32 elem_size = sizeof(struct stack_map_bucket) + smap->map.value_size; + u64 elem_size = sizeof(struct stack_map_bucket) + (u64)smap->map.value_size; }</pre>	
Phase 1: Multi-perspective Reasoning	
 I am Alice. I follow Deductive Reasoning .	Reasoning Summary: Formal reasoning based on resource management rules (CERT MEM34-C) → confirms correct deallocation on all error paths, thus no memory leaks. Vulnerability: No
 I am Bob. I follow Inductive Reasoning .	Reasoning Summary: Pattern-matching via historical vulnerability-fix pairs → snippet aligned with benign allocation patterns, thus no flaw inferred, assuming input validation upstream. Vulnerability: No
 I am Carol. I follow Abductive Reasoning .	Reasoning Summary: Starts with hypothesis of heap overflow → best explanation: integer overflow in size calculation, caused by user-controlled inputs, leading to undersized allocation and memory corruption. Vulnerability: Yes
Phase 2: Collaborative Debate	
 Now, I will review others' reasoning	Reasoning Summary: I must change my initial conclusion. Incorporating new premise (arithmetic overflow) → invalidates initial assumption of safe arithmetic; proves out-of-bounds writes violating memory safety. Vulnerability: Yes
 Now, I will review others' reasoning	Reasoning Summary: I must change my initial conclusion. I reviewed the pattern related to CWE-190 → the code contained an unchecked multiplication in an assignment, thus overturning the previous safety inference. Vulnerability: Yes
 Now, I will review others' reasoning	Reasoning Summary: I maintain my initial conclusion. Maintains the same hypothesis and explanation → confirms overflow-induced corruption as the most plausible root cause. Vulnerability: Yes

Fig. 5. Case study of the REASONVUL for CVE-2021-41864.

structured debate, enables REASONVUL to detect subtle vulnerabilities that might be overlooked by any single reasoning method or by simple aggregation approaches such as majority voting. The condensed transcript of the agent interactions is depicted in Fig. 5, with concise reasoning summaries generated by the LLM for clarity.

In the Multi-perspective Reasoning phase, each agent independently analyzes the code, leveraging its unique reasoning paradigm. The Deductive Agent, applying the formal security principle of resource management (CERT MEM34-C), confirms that the function's control flow correctly deallocates memory on all error paths, concluding that the code is compliant with this specific rule and shows no evidence of memory leaks. However, this focus on memory management overlooks the subtle arithmetic boundary violation causing the integer overflow. The Inductive Agent, relying on pattern-matching against historical vulnerability data from the ReposVul dataset, does not detect the issue, as the specific overflow pattern is not well-represented in its knowledge base. Only the Abductive Agent, through hypothesis-driven reasoning, identifies the vulnerability by postulating a failure scenario involving excessive user input that could trigger an integer overflow, leading to an out-of-bounds access. This divergence highlights a key insight: relying on a single reasoning paradigm is insufficient for detecting the complex and diverse vulnerabilities that arise in real-world scenarios.

The Collaborative Debate phase is pivotal in synthesizing these diverse perspectives and resolving the initial 2-to-1 conflict (Deductive and Inductive Agents voting "benign" versus Abductive Agent voting "vulnerable"). As depicted in Fig. 5, a simple majority voting approach at this stage would have erroneously classified the code as benign, overlooking the vulnerability. Instead, REASONVUL's structured debate mechanism enables dynamic error correction. The Abductive Agent presents its hypothesis-driven evidence, articulating a plausible attack scenario that highlights the overflow risk. This prompts the Deductive Agent to re-evaluate the code beyond CERT MEM34-C, recognizing a previously overlooked violation in arithmetic safety principles. Similarly, when confronted with the abductive hypothesis, the inductive agent re-evaluates its pattern-matching approach, acknowledges that it previously overlooked this vulnerability pattern, and adjusts its stance to align with the new evidence. Through this iterative dialogue, all agents are ultimately able to correctly

identify the vulnerability. To further demonstrate the effectiveness of collaborative debate, we compared its performance with majority voting in resolving conflicting judgments. Across 542 samples with initial conflicts in the PrimeVul dataset, majority voting successfully resolved only 49 cases, accounting for a mere 9.04%. In contrast, the collaborative debate mechanism correctly resolved 389 cases, representing 71.77%. This striking disparity underscores the capability of the debate mechanism to emulate the critical, multi-perspective reasoning of human expert panels, enabling agents to challenge and refine their analyses through evidence-based discourse.

This case study, coupled with the quantitative comparison, highlights two key strengths of REASONVUL. First, multi-perspective reasoning is essential, as the complementary nature of deductive, inductive, and abductive approaches ensures that diverse aspects of complex vulnerabilities are captured, which cannot be achieved by a single paradigm alone. Second, the collaborative debate mechanism turns initial differences in opinion into a chance to improve detection accuracy, facilitating a dynamic process of discovery and error correction that significantly outperforms simplistic aggregation methods like majority voting. Compared to single-paradigm methods or basic ensemble approaches, REASONVUL's integration of multi-perspective reasoning and structured debate substantially enhances the accuracy of vulnerability detection.

7.2 Analysis of Collaborative Synergy

To quantify the synergy generated by our framework, this section analyzes how the collaborative debate synthesizes the diverse outputs from the individual Deductive, Inductive, and Abductive agents. The analysis, based on the PrimeVul dataset evaluation, reveals that the debate mechanism is crucial for integrating complementary perspectives into a unified and more accurate judgment.

Fig. 6 illustrates the complementarity among the agents' initial findings. Before the debate, the deductive, abductive, and inductive agents independently detected 283, 293, and 270 vulnerabilities, respectively, with their intersection limited to just 133 vulnerabilities. This overlap confirms that each reasoning paradigm possesses unique strengths but is also hindered by notable blind spots. After the debate, however, REASONVUL successfully identifies a total of 347 vulnerabilities, of which 214 cases represent instances where the debate effectively resolved initial disagreements. This demonstrates that the debate mechanism effectively transforms fragmented and often conflicting initial analyses into a consistent and far more comprehensive final result. REASONVUL outperforms the individual agents by detecting 64, 54, and 77 additional vulnerabilities compared to the deductive, abductive, and inductive agents, respectively. This enhanced coverage highlights the framework's capacity to leverage the collective intelligence of its agents, uncovering subtle flaws that elude single-paradigm approaches. The significant increase in detected vulnerabilities demonstrates the power of cognitive synergy facilitated by structured debate.

Nevertheless, the analysis also reveals avenues for future enhancement. The Deductive and Inductive agents alone identified 25 and 12 vulnerabilities, respectively, which were not part of REASONVUL's final verdict. This indicates that the current debate process may sometimes over-prioritize certain lines of reasoning, potentially missing some valid rule-based or pattern-based

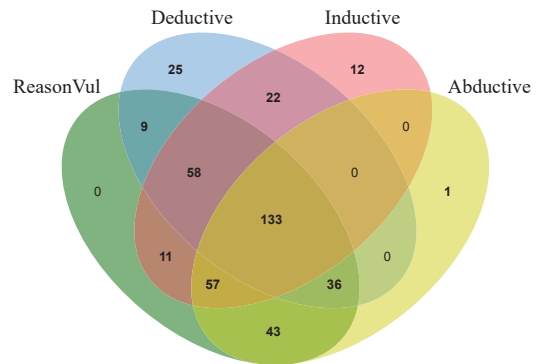


Fig. 6. The overlaps of the vulnerabilities detected by different approaches.

vulnerabilities. Future work could aim to refine the debate dynamics to ensure a more balanced synthesis of all expert insights, further strengthening the framework's detection capabilities.

7.3 Threats to Validity

External Threats. Our study faces several external threats. First, the choice of dataset language may introduce selection bias. REASONVUL performs excellently on C/C++ datasets, but its effectiveness in other programming languages (such as Python, Java, or JavaScript) has yet to be validated. Although our multi-agent debate framework is language-agnostic, the prompts and reasoning patterns for different languages may need adjustments. Future work will evaluate cross-language generalizability. Secondly, there may be selection bias in the LLMs used in both REASONVUL and the baselines. Our experiments selected seven mainstream LLMs due to their strong performance on code-related tasks and their availability for research [18, 73]. However, the rapid development of LLMs may lead to new models performing differently in vulnerability detection, limiting the applicability of our findings to models that were not evaluated. We mitigate this by selecting code-proficient models, with plans for broader evaluations.

Internal Threats. One potential threat is the accuracy of the baseline methods we have re-implemented. Differences in implementation details (e.g., hyperparameter settings) could affect performance comparisons. To mitigate this, we use official code and original configurations to ensure fairness. Another internal threat arises from the inherent randomness in the outputs of LLMs. To address this, we set the temperature to 0 to make the results more deterministic. Finally, the selection of optimal LLMs for each agent based on the PairVul dataset poses a potential risk of data leakage regarding the PrimeVul test set. To address this, we analyzed the data distribution differences between the two datasets. The PrimeVul covers more than 100 CWE types drawn from hundreds of large projects, whereas PairVul is mainly restricted to 5 CWE types from the Linux kernel. The calculated token-level Jensen–Shannon divergence between the two datasets is 0.43, indicating a substantial distributional shift. This quantitative evidence confirms that using PairVul for model selection does not leak information from the PrimeVul test set, ensuring the validity of our evaluation.

8 Conclusion

In this paper, we presented REASONVUL, a multi-perspective reasoning framework that advances automated vulnerability detection by integrating deductive, inductive, and abductive reasoning paradigms within a collaborative multi-agent system. Our approach achieves cognitive synergy through two stages, i.e., independent reasoning and collaborative debate, to resolve conflicts and identify subtle gaps that are overlooked by single-paradigm methods. Our experiments demonstrate that REASONVUL significantly outperforms state-of-the-art vulnerability detection methods, achieving a PairAcc of 40.00% and an F1-score of 72.52% on the PrimeVul dataset. This research highlights the critical importance of cognitive diversity and structured debate in vulnerability detection, offering a promising approach to improving the security and reliability of software systems across diverse real-world scenarios.

9 Data Availability

All the code and data are publicly available at <https://zenodo.org/records/17106263>.

Acknowledgments

This work was supported by the National Key R&D Program of China (Grant No. 2024YFB4506200), the Natural Science Foundation of Hunan Province (Grant No. 2026JJ40060), and the Research Foundation from NUDT (Grant No. ZK24-05).

References

- [1] Andrei Arusoae, Stefan Ciobăca, Vlad Craciun, Dragos Gavrilut, and Dorel Lucanu. 2017. A Comparison of Open-Source Static Analysis Tools for Vulnerability Detection in C/C++ Code. In *2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*. 161–168. doi:10.1109/SYNASC.2017.00035
- [2] Canan Batur Şahin and Laith Abualigah. 2021. A novel deep learning-based feature selection model for improving the static analysis of vulnerability detection. *Neural Computing and Applications* 33, 20 (2021), 14049–14067. doi:10.1007/s00521-021-06047-x
- [3] Sicong Cao, Xiaobing Sun, Lili Bo, Ying Wei, and Bin Li. 2021. Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection. *Information and Software Technology* 136 (2021), 106576. doi:10.1016/j.infsof.2021.106576
- [4] Sicong Cao, Xiaobing Sun, Lili Bo, Rongxin Wu, Bin Li, and Chuanqi Tao. 2022. MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1456–1468. doi:10.1145/3510003.3510219
- [5] Sicong Cao, Xiaobing Sun, Xiaoxue Wu, David Lo, Lili Bo, Bin Li, and Wei Liu. 2024. Coca: Improving and explaining graph neural network-based vulnerability detection systems. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639168
- [6] Ira Ceka, Feitong Qiao, Anik Dey, Aastha Valecha, Gail Kaiser, and Baishakhi Ray. 2024. Can llm prompting serve as a proxy for static analysis in vulnerability detection. *arXiv preprint arXiv:2412.12039* (2024).
- [7] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. 2021. DeepWukong: Statically Detecting Software Vulnerabilities Using Deep Graph Neural Network. *ACM Trans. Softw. Eng. Methodol.* 30, 3, Article 38 (April 2021), 33 pages. doi:10.1145/3436877
- [8] Seyed Shayan Daneshvar, Yu Nong, Xu Yang, Shaowei Wang, and Haipeng Cai. 2025. VulScribeR: Exploring RAG-based Vulnerability Augmentation with LLMs. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3760775
- [9] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability Detection with Code Language Models: How Far Are We?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 469–481. doi:10.1109/ICSE55347.2025.00038
- [10] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, et al. 2024. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag. *arXiv preprint arXiv:2406.11147* (2024).
- [11] Andrew Estornell and Yang Liu. 2024. Multi-LLM Debate: Framework, Principals, and Interventions. 37 (2024), 28938–28964. doi:10.52202/079017-0911
- [12] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [13] Michael Fu and Chakkrit Tantithamthavorn. 2022. Linevul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 608–620. doi:10.1145/3524842.3528452
- [14] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. Chatgpt for vulnerability detection, classification, and repair: How far are we?. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 632–636. doi:10.1109/apsec60848.2023.00085
- [15] Zeyu Gao, Hao Wang, Yuchen Zhou, Wenyu Zhu, and Chao Zhang. 2023. How far have we gone in vulnerability detection using large language models. *arXiv preprint arXiv:2311.12420* (2023).
- [16] GNU. 2025. Gnu cflow: analyzing a collection of c source files, charting control flow within the program. (2025).
- [17] Nima Shiri Harzevili, Alvine Boaye Belle, Junjie Wang, Song Wang, Zhen Ming, Nachiappan Nagappan, et al. 2023. A survey on automated software vulnerability detection using machine learning and deep learning. *arXiv preprint arXiv:2306.11673* (2023).
- [18] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. doi:10.1145/3712003
- [19] Kaiyu He, Mian Zhang, Shuo Yan, Peilin Wu, and Zhiyu Zoey Chen. 2024. Idea: Enhancing the rule learning ability of large language model agent through induction, deduction, and abduction. *arXiv preprint arXiv:2408.10455* (2024).
- [20] Sihao Hu, Tiansheng Huang, Fatih İlhan, Selim Furkan Tekin, and Ling Liu. 2023. Large language model-powered smart contract vulnerability detection: New perspectives. In *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. IEEE, 297–306.
- [21] Larry Huynh, Yinghao Zhang, Djimon Jayasundera, Woojin Jeon, Hyoungshick Kim, Tingting Bi, and Jin B. Hong. 2025. Detecting Code Vulnerabilities using LLMs. In *2025 55th Annual IEEE/IFIP International Conference on Dependable*

- Systems and Networks (DSN)*. 401–414. doi:10.1109/DSN64029.2025.00047
- [22] Ridhi Jain, Nicole Gervasoni, Mthandazo Ndhlovu, and Sanjay Rawat. 2023. A code centric evaluation of c/c++ vulnerability datasets for deep learning based vulnerability detection techniques. In *Proceedings of the 16th Innovations in Software Engineering Conference*. 1–10.
 - [23] Yuan Jiang, Yujian Zhang, Xiaohong Su, Christoph Treude, and Tiantian Wang. 2024. Stagedvulbert: Multi-granular vulnerability detection with a novel pre-trained code model. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3493245
 - [24] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2023. Loogle: Can long-context language models understand long contexts? *arXiv preprint arXiv:2311.04939* (2023).
 - [25] Yue Li, Xiao Li, Hao Wu, Minghui Xu, Yue Zhang, Xiuzhen Cheng, Fengyuan Xu, and Sheng Zhong. 2025. Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask. *arXiv preprint arXiv:2504.13474* (2025).
 - [26] Ziyang Li, Saikat Dutta, and Mayur Naik. 2024. IRIS: LLM-assisted static analysis for detecting security vulnerabilities. *arXiv preprint arXiv:2405.17238* (2024).
 - [27] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018).
 - [28] RongTao Liao, XueHu Yan, and KaiLong Zhu. 2025. KLRAG: Deep Learning Library Vulnerability Detection via Knowledge-Level RAG. In *International Conference on Intelligent Computing*. Springer, 443–456. doi:10.1007/978-981-96-9958-2_36
 - [29] Bo Lin, Shangwen Wang, Yihao Qin, Liqian Chen, and Xiaoguang Mao. 2025. Give LLMs a Security Course: Securing Retrieval-Augmented Code Generation via Knowledge Injection. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3719027.3765049
 - [30] GuanJun Lin, Sheng Wen, Qing-Long Han, Jun Zhang, and Yang Xiang. 2020. Software Vulnerability Detection Using Deep Neural Networks: A Survey. *Proc. IEEE* 108, 10 (2020), 1825–1848. doi:10.1109/JPROC.2020.2993293
 - [31] Stephan Lipp, Sebastian Banescu, and Alexander Pretschner. 2022. An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, South Korea) (ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 544–555. doi:10.1145/3533767.3534380
 - [32] Yuhan Liu, Yuxuan Liu, Xiaoqing Zhang, Xiuying Chen, and Rui Yan. 2025. The Truth Becomes Clearer Through Debate! Multi-Agent Systems with Large Language Models Unmask Fake News. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (Padua, Italy) (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 504–514. doi:10.1145/3726302.3730092
 - [33] Yisroel Mirsky, George Macon, Michael Brown, Carter Yagemann, Matthew Pruett, Evan Downing, Sukarno Mertoguno, and Wenke Lee. 2023. VulChecker: Graph-based Vulnerability Localization in Source Code. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 6557–6574.
 - [34] Van Nguyen, Trung Le, Chakkrit Tantithamthavorn, John Grundy, and Dinh Phung. 2024. Deep domain adaptation with max-margin principle for cross-project imbalanced software vulnerability detection. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–34. doi:10.1145/3664602
 - [35] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230* (2024).
 - [36] Chitu Okoli. 2023. Inductive, abductive and deductive theorising. *International Journal of Management Concepts and Philosophy* 16, 3 (2023), 302–316. doi:10.1504/IJMCP.2023.131769
 - [37] Xin Peng, Jieren Cheng, Xiangyan Tang, Jingxin Liu, and Jiahua Wu. 2023. Dual contrastive learning network for graph clustering. *IEEE Transactions on Neural Networks and Learning Systems* 35, 8 (2023), 10846–10856. doi:10.1109/TNNLS.2023.3244397
 - [38] Xin Peng, Jieren Cheng, Xiangyan Tang, Bin Zhang, and Wenxuan Tu. 2024. Multi-view graph imputation network. *Information Fusion* 102 (2024), 102024. doi:10.1016/j.inffus.2023.102024
 - [39] Xin Peng, Shangwen Wang, Yihao Qin, Bo Lin, Liqian Chen, Jieren Cheng, and Xiaoguang Mao. 2025. Keep It Simple: Self-Adaptive Code Graph Simplification for Accurate Vulnerability Detection. *IEEE Transactions on Software Engineering* 51, 10 (2025), 2744–2763. doi:10.1109/TSE.2025.3593515
 - [40] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. 2024. Towards Causal Deep Learning for Vulnerability Detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery. doi:10.1145/3597503.3639170
 - [41] Shahriyar Zaman Ridoy, Md Shazzad Hossain Shaon, Alfredo Cuzzocrea, and Mst Shapna Akter. 2024. EnStack: An Ensemble Stacking Framework of Large Language Models for Enhanced Vulnerability Detection in Source Code. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 6356–6364. doi:10.1109/BigData62323.2024.10825609

- [42] Niklas Risse, Jing Liu, and Marcel Böhme. 2025. Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 388–410. doi:10.1145/3728887
- [43] Robert C Seacord. 2008. The CERT C secure coding standard. *Pearson Education* (2008).
- [44] Miaomiao Shao and Yuxin Ding. 2024. FVD-DPM: Fine-grained Vulnerability Detection via Conditional Diffusion Probabilistic Models. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 7375–7392.
- [45] Yu Sheng, Wanting Wen, Linjing Li, and Daniel Zeng. 2025. Evaluating Generalization Capability of Language Models across Abductive, Deductive and Inductive Logical Reasoning. In *Proceedings of the 31st International Conference on Computational Linguistics*. 4945–4957.
- [46] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2025. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *arXiv preprint arXiv:2502.07049* (2025).
- [47] Samiha Shimm, Ashiqur Rahman, Mohan Gadde, Hamed Okhravi, and Mona Rahimi. 2024. VulSim: Leveraging Similarity of Multi-Dimensional Neighbor Embeddings for Vulnerability Detection. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 1777–1794.
- [48] Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow Analysis-Inspired Deep Learning for Efficient Vulnerability Detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery. doi:10.1145/3597503.3623345
- [49] Benjamin Steenhoek, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An Empirical Study of Deep Learning Models for Vulnerability Detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2237–2248. doi:10.1109/ICSE48619.2023.00188
- [50] Benjamin Steenhoek, Md Mahbubur Rahman, Monoshi Kumar Roy, Mirza Sanjida Alam, Hengbo Tong, Swarna Das, Earl T Barr, and Wei Le. 2024. To err is machine: Vulnerability detection challenges llm reasoning. *arXiv preprint arXiv:2403.17218* (2024).
- [51] Saba Sturua, Isabelle Mohr, Mohammad Kalim Akram, Michael Günther, Bo Wang, Markus Krimmel, Feng Wang, Georgios Mastrapas, Andreas Koukounas, Nan Wang, et al. 2024. jina-embeddings-v3: Multilingual embeddings with task lora. *arXiv preprint arXiv:2409.10173* (2024).
- [52] Shangwen Wang, Bo Lin, Liqian Chen, and Xiaoguang Mao. 2025. Divide-and-Conquer: Automating Code Revisions via Localization-and-Revision. 34, 3 (2025). doi:10.1145/3697013
- [53] Xincheng Wang, Ruida Hu, Cuiyun Gao, Xin-Cheng Wen, Yujia Chen, and Qing Liao. 2024. Reposvul: A repository-level high-quality vulnerability dataset. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 472–483. doi:10.1145/3639478.3647634
- [54] Zhiyuan Wei, Jing Sun, Yuqiang Sun, Ye Liu, Daoyuan Wu, Zijian Zhang, Xianhao Zhang, Meng Li, Yang Liu, Chunmiao Li, et al. 2025. Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems. *IEEE Transactions on Software Engineering* (2025). doi:10.1109/TSE.2025.3597319
- [55] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M Zhang, and Qing Liao. 2023. Vulnerability detection with graph simplification and enhanced graph representation learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2275–2286. doi:10.1109/ICSE48619.2023.00191
- [56] Xin-Cheng Wen, Xincheng Wang, Yujia Chen, Ruida Hu, David Lo, and Cuiyun Gao. 2024. Vuleval: Towards repository-level evaluation of software vulnerability detection. *arXiv preprint arXiv:2404.15596* (2024).
- [57] Xin-Cheng Wen, Jiaxin Ye, Cuiyun Gao, Lianwei Wu, and Qing Liao. 2024. EvalSVA: Multi-Agent Evaluators for Next-Gen Software Vulnerability Assessment. *arXiv preprint arXiv:2501.14737* (2024).
- [58] Ratnadira Widayarsi, Martin Weyssow, Ivana Clairen Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. 2025. Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection using LLM-Based Agents. *arXiv preprint arXiv:2505.10961* (2025).
- [59] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. 2022. Vulcnn: An image-inspired scalable vulnerability detection system. In *Proceedings of the 44th International Conference on Software Engineering*. 2365–2376. doi:10.1145/3510003.3510229
- [60] Yuying Xia, Haijian Shao, and Xing Deng. 2024. Vulcobert: A codebert-based system for source code vulnerability detection. In *Proceedings of the 2024 international conference on generative artificial intelligence and information security*. 249–252.
- [61] Xu Yang, Shaowei Wang, Yi Li, and Shaohua Wang. 2023. Does data sampling improve deep learning-based vulnerability detection? yeas! and nays!. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2287–2298. doi:10.1109/ICSE48619.2023.00192
- [62] Xu Yang, Shaowei Wang, Jiayuan Zhou, and Wenhan Zhu. 2025. One-for-All Does Not Work! Enhancing Vulnerability Detection by Mixture-of-Experts (MoE). *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 446–464. doi:10.1145/3715736
- [63] Alperen Yildiz, Sin G Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil M Divakaran. 2025. Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories. *arXiv preprint arXiv:2503.03586* (2025).

- [64] Xin Yin, Chao Ni, and Shaohua Wang. 2024. Multitask-based evaluation of open-source llm on software vulnerability. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3470333
- [65] Jeffy Yu. 2024. Retrieval Augmented Generation Integrated Large Language Models in Smart Contract Vulnerability Detection. *arXiv preprint arXiv:2407.14838* (2024).
- [66] Chenyuan Zhang, Hao Liu, Jiutian Zeng, Kejing Yang, Yuhong Li, and Hui Li. 2024. Prompt-enhanced software vulnerability detection using chatgpt. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 276–277. doi:10.1145/3639478.3643065
- [67] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Xudong Liu, Chunming Hu, and Yang Liu. 2023. Detecting condition-related bugs with control flow graph neural network. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1370–1382.
- [68] Tiehua Zhang, Rui Xu, Jianping Zhang, Yuze Liu, Xin Chen, Jun Yin, and Xi Zheng. 2024. DSHGT: Dual-Supervisors Heterogeneous Graph Transformer—A Pioneer Study of Using Heterogeneous Graph Learning for Detecting Software Vulnerabilities. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–31. doi:10.1145/3674729
- [69] Yu Zhao, Lina Gong, Zhiqiu Huang, Yongwei Wang, Mingqiang Wei, and Fei Wu. 2024. Coding-ptms: How to find optimal code pre-trained models for code embedding in vulnerability detection?. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1732–1744. doi:10.1145/3691620.3695539
- [70] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–31. doi:10.1145/3708522
- [71] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 47–51. doi:10.1145/3639476.3639762
- [72] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems* 32 (2019).
- [73] Kangchen Zhu, Zhiliang Tian, Shangwen Wang, Weiguo Chen, Zixuan Dong, Mingyue Leng, and Xiaoguang Mao. 2025. MiSum: Multi-modality Heterogeneous Code Graph Learning for Multi-intent Binary Code Summarization. *Proc. ACM Softw. Eng.* 2, FSE (2025). doi:10.1145/3715780
- [74] Yuan Zhuang, Zhengguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He. 2021. Smart contract vulnerability detection using graph neural networks. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence* (Yokohama, Yokohama, Japan) (*IJCAI'20*). Article 454, 8 pages. doi:10.24963/ijcai.2020/454
- [75] Arastoo Zibaeirad and Marco Vieira. 2024. Vulnllmeval: A framework for evaluating large language models in software vulnerability detection and patching. *arXiv preprint arXiv:2409.10756* (2024).
- [76] Arastoo Zibaeirad and Marco Vieira. 2025. Reasoning with llms for zero-shot vulnerability detection. *arXiv preprint arXiv:2503.17885* (2025).

Received 2025-09-04; accepted 2026-03-24